

Stock Market Predictions

Phase 1: Problem Understanding & Industry Analysis

1. Requirement Gathering

The goal is to help investors and analysts manage stock data inside Salesforce.

- Store stock details (company name, ticker, sector, price).
- Allow investors to track their portfolios (buy/sell transactions).
- Show predictions (future price trends).
- Provide reports and dashboards (gains/losses, prediction accuracy).
- Send alerts when stock crosses certain thresholds.

2. Stakeholder Analysis

- Investors (End Users) → Want to see their portfolio and stock predictions.
- Financial Analysts → Need dashboards to compare stock performance.
- Admin/Developer → Setup objects, flows, automation, and integrations.
- Executives → Want summary dashboards for decision making.

3. Business Process Mapping

- Current Process (outside Salesforce):
 - Investors use multiple apps like Yahoo Finance, Excel, TradingView.
 - Predictions are either manual or from third-party websites.
 - No single system to combine portfolio + real-time data + predictions.
- Proposed Process (inside Salesforce):
 - Salesforce fetches stock data from an external API.
 - Data is stored in custom objects (Stock, Portfolio, Transaction, Prediction).

- Automation updates portfolio value and prediction records.
- Reports and dashboards show insights (Top gainers, Loss %).
- Alerts sent to investors when price moves significantly.

4. Industry-Specific Use Case Analysis

- Retail Investors → Manage their personal stock portfolios inside Salesforce.
- Investment Firms → Track multiple client portfolios in one platform.
- Wealth Management Companies → Give AI-based recommendations using predictions.
-  Example: If an investor has 10 shares of Company, Salesforce shows:
- Current Value = Quantity × Price.
- Predicted Next Price = +5%.
- System sends an alert: “Your Company holdings may rise 5% tomorrow.”

5. AppExchange Exploration

- Existing apps provide stock tickers or financial dashboards.
- But there is no complete prediction-focused app on AppExchange.
- This project fills that gap by combining portfolio tracking + prediction + alerts.

Phase 2: Org Setup & Configuration

1. Salesforce Edition

- Use Developer Edition → It's free and provides all the features required (custom objects, automation, API integration).
- Suitable for learning and building a POC (Proof of Concept).

2. Company Profile Setup

- Fiscal Year → Set from January to December (to align with global stock reporting).
- Default Currency → Indian Rupee (₹) or USD (\$), depending on stock market focus.
- Timezone → Align with the stock exchange (e.g., GMT+5:30 for India).

3. Business Hours & Holidays

- Set Business Hours → 9:30 AM – 3:30 PM (Indian NSE/BSE market).
- Configure Holidays → Official stock market holidays (e.g., Diwali, Independence Day, Republic Day).
- Purpose → Helps with SLA calculations and alerts only during market hours.

4. User Setup & Licenses

- Users:
Admin User → Full control (manages configurations).
- Investor User → Restricted access (views only their portfolio).
Assign Salesforce Platform License to Investor user (cost-effective in real scenario).

5. Profiles

- Admin Profile → CRUD (Create, Read, Update, Delete) on all objects.
- Investor Profile → Read-only access to Stocks & Predictions, Read/Write to their Portfolio.

6. Roles

- Admin Role → Higher in hierarchy, can see all data.
- Investor Role → Lower in hierarchy, can only see their own records.

7. Permission Sets

- Create "API Access Permission Set" → Grants API access for stock data integration.
- Assign this only to Admin.

8. Org-Wide Defaults (OWD)

- Portfolio & Transactions → Private (only owner can see).
- Stock → Public Read-Only (everyone can see stock prices).
- Prediction → Controlled by Parent (linked to Stock).

9. Sharing Rules

- Share Portfolios only with the specific investor.
- Admin can see everything, but one investor cannot see another's portfolio.

10. Login Access Policies

- Enable IP Restrictions for Admin login (e.g., office network only).
- Allow investors to log in from anywhere.

11. Dev Org Setup

- Create Developer Org (from Salesforce.com → Free Signup).
- Install sample financial datasets (CSV imports).

12. Sandbox Usage

Use Developer Sandbox for:

- Testing API integrations (stock price API).
- Testing automation (flows, triggers).

- Deploy to production only after validation.

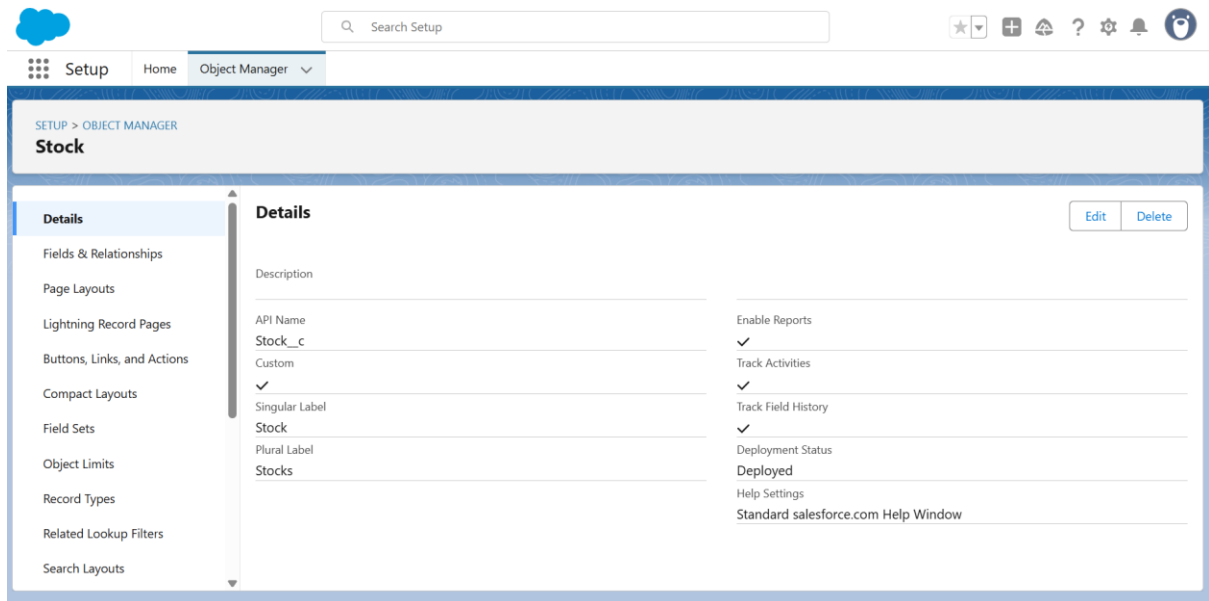
13. Deployment Basics

- Use **Change Sets** for deployment (simple beginner method).
- Advanced option: Use **VS Code** + **SFDX** for metadata deployment.

Phase 3: Data Modeling & Relationships

1. Custom Objects & Key Fields

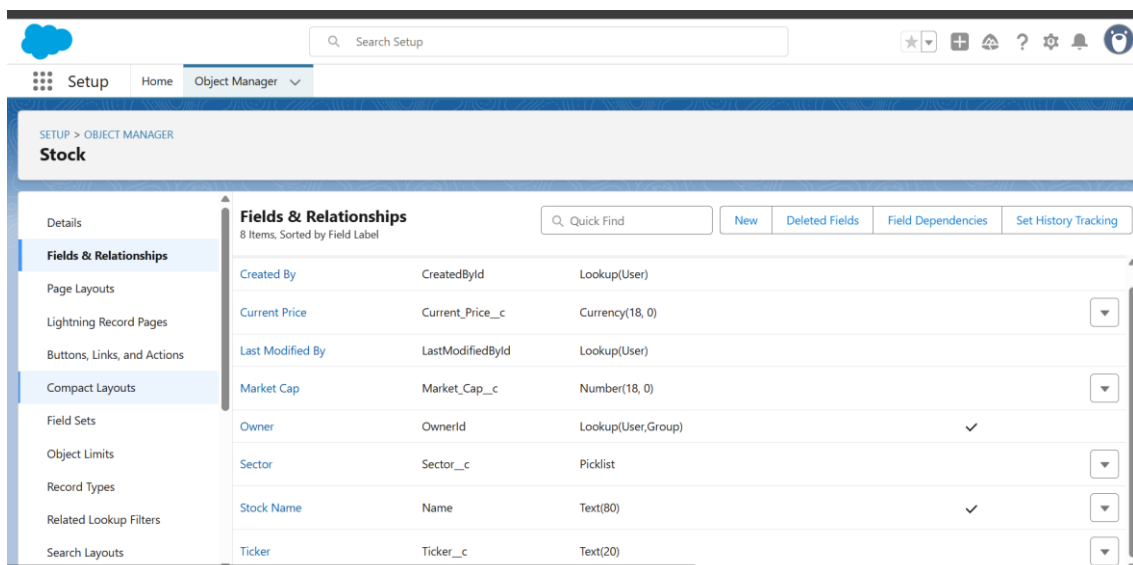
 Stock__c



The screenshot shows the Salesforce Setup interface for the custom object 'Stock__c'. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, etc. The main content area is titled 'Details' and shows various configuration options for the object, including API Name (Stock__c), Singular Label (Stock), Plural Label (Stocks), and various tracking and reporting settings.

Fields:

- Ticker__c (Text, e.g., TCS, INFY)
- Sector__c (Picklist: IT, Pharma, Banking, etc.)
- Current_Price__c (Currency)
- Market_Cap__c (Number)



The screenshot shows the 'Fields & Relationships' section for the 'Stock__c' object. It displays a list of 8 fields, sorted by Field Label. The fields are: Created By, Current Price, Last Modified By, Market Cap, Owner, Sector, Stock Name, and Ticker. Each field entry shows its name, API Name, and data type. The 'Current Price' and 'Market Cap' fields are highlighted in blue.

Field Name	API Name	Data Type
Created By	CreatedBy	Lookup(User)
Current Price	Current_Price__c	Currency(18, 0)
Last Modified By	LastModifiedBy	Lookup(User)
Market Cap	Market_Cap__c	Number(18, 0)
Owner	OwnerId	Lookup(User, Group)
Sector	Sector__c	Picklist
Stock Name	Name	Text(80)
Ticker	Ticker__c	Text(20)

Portfolio__c

The screenshot shows the Salesforce Setup interface for the Portfolio__c object. The left sidebar contains a navigation menu with options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Search Layouts. The main content area is titled 'Portfolio' and shows the 'Details' tab. The 'Details' section includes a description field, API Name (Portfolio__c), Custom Singular Label (Portfolio), and Plural Label (Portfolios). On the right, there are checkboxes for 'Enable Reports' (checked), 'Track Activities' (checked), 'Track Field History' (checked), and 'Deployment Status' (Deployed). At the bottom right, there are links for 'Help Settings' and 'Standard salesforce.com Help Window'. 'Edit' and 'Delete' buttons are located at the top right of the details section.

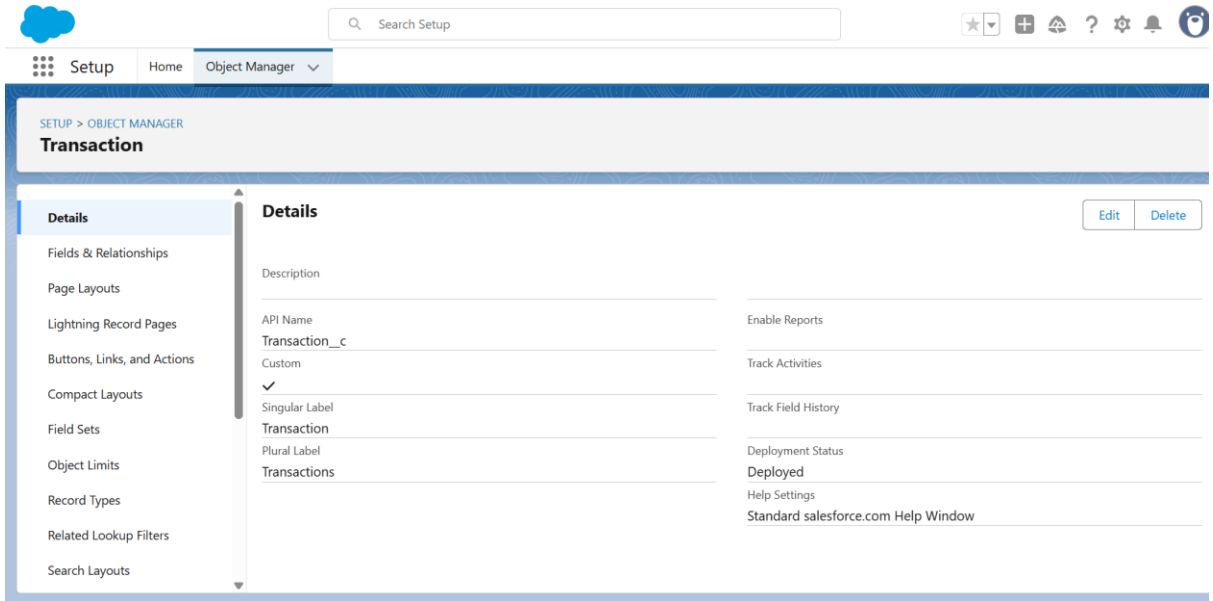
Fields:

- Investor__c (Lookup → User)
- Total_Value__c (Currency, roll-up from Transactions)
- Risk_Profile__c (Picklist: Low, Medium, High)

The screenshot shows the Salesforce Setup interface for the Portfolio__c object, specifically the 'Fields & Relationships' section. The left sidebar is the same as the previous screenshot. The main content area is titled 'Portfolio' and shows the 'Fields & Relationships' tab. The section includes a search bar 'Q, Quick Find' and buttons for 'New', 'Deleted Fields', 'Field Dependencies', and 'Set History Tracking'. Below this is a table with 7 items, sorted by Field Label. The table has columns: FIELD LABEL, FIELD NAME, DATA TYPE, CONTROLLING FIELD, and INDEXED. The rows are: Created By (CreatedBy, Lookup(User)), Investor (Investor__c, Lookup(User), indexed), Last Modified By (LastModifiedBy, Lookup(User)), Owner (OwnerId, Lookup(User,Group), indexed), Portfolio Name (Name, Text(80), indexed), Risk Profile (Risk_Profile__c, Picklist), and Total Value (Total_Value__c, Currency(18, 0)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedBy	Lookup(User)		
Investor	Investor__c	Lookup(User)		✓
Last Modified By	LastModifiedBy	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Portfolio Name	Name	Text(80)		✓
Risk Profile	Risk_Profile__c	Picklist		
Total Value	Total_Value__c	Currency(18, 0)		

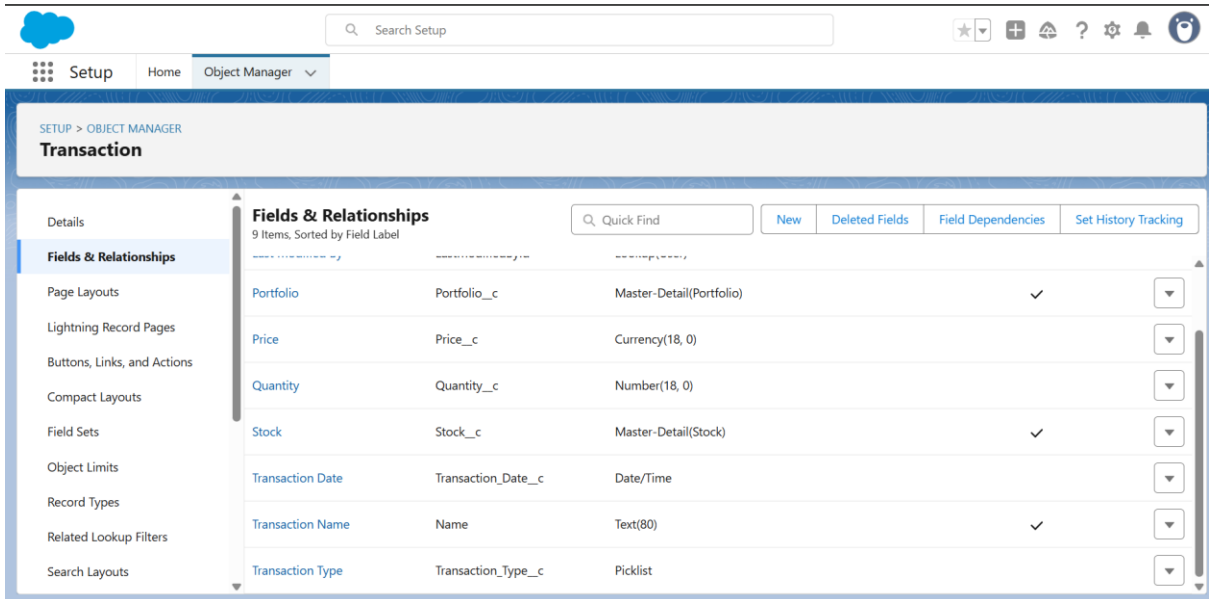
Transaction__c (Junction between Stock & Portfolio)



The screenshot shows the Salesforce Setup interface for the Transaction__c object. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, etc. The main content area is titled 'Transaction' and shows the 'Details' tab. The details section includes fields for Description, API Name (Transaction__c), Custom (checked), Singular Label (Transaction), Plural Label (Transactions), Enable Reports, Track Activities, Track Field History, Deployment Status (Deployed), and Help Settings (Standard salesforce.com Help Window). There are 'Edit' and 'Delete' buttons in the top right corner.

Fields:

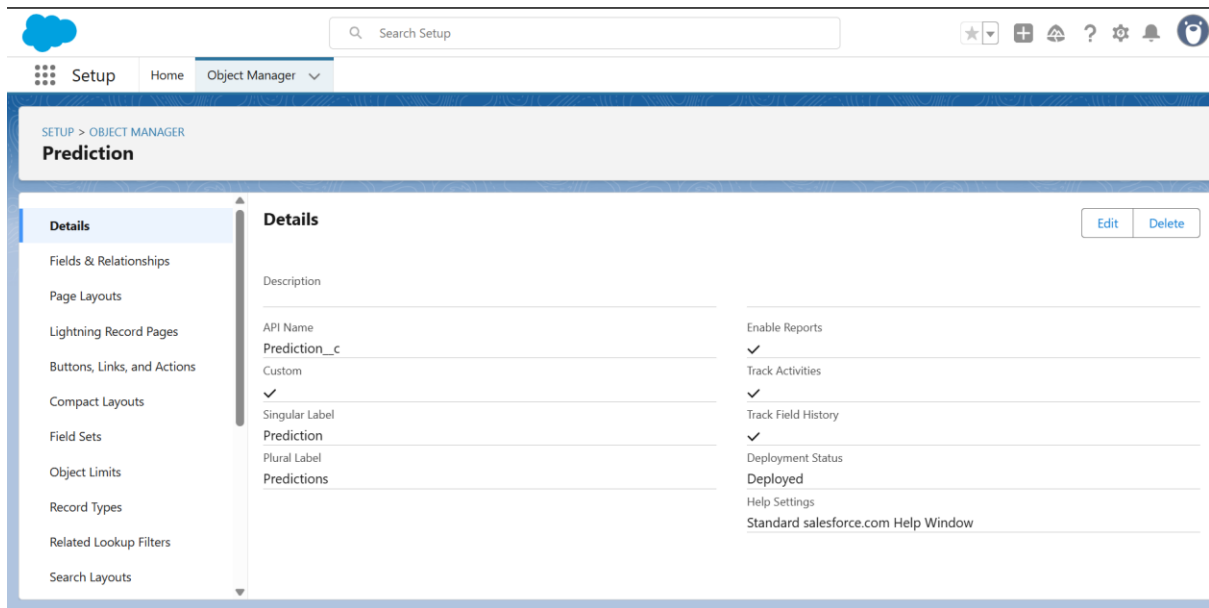
- Portfolio__c (Master-Detail)
- Stock__c (Master-Detail)
- Transaction_Type__c (Picklist: Buy, Sell)
- Quantity__c (Number)
- Price__c (Currency)
- Transaction_Date__c (Date/Time)



The screenshot shows the Salesforce Setup interface for the Transaction__c object, specifically the 'Fields & Relationships' tab. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, etc. The main content area is titled 'Transaction' and shows the 'Fields & Relationships' section. It includes a 'Quick Find' search bar and buttons for 'New', 'Deleted Fields', 'Field Dependencies', and 'Set History Tracking'. The table below lists the fields and their relationships:

Field Label	Field Name	Field Type	Relationship	Checkmark	Dropdown
Portfolio	Portfolio__c	Master-Detail(Portfolio)		✓	▼
Price	Price__c	Currency(18, 0)			▼
Quantity	Quantity__c	Number(18, 0)			▼
Stock	Stock__c	Master-Detail(Stock)		✓	▼
Transaction Date	Transaction_Date__c	Date/Time			▼
Transaction Name	Name	Text(80)		✓	▼
Transaction Type	Transaction_Type__c	Picklist			▼

Prediction__c



Setup > OBJECT MANAGER

Prediction

Details

Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Details

Description

API Name
Prediction__c

Custom

Singular Label
Prediction

Plural Label
Predictions

Enable Reports
✓

Track Activities
✓

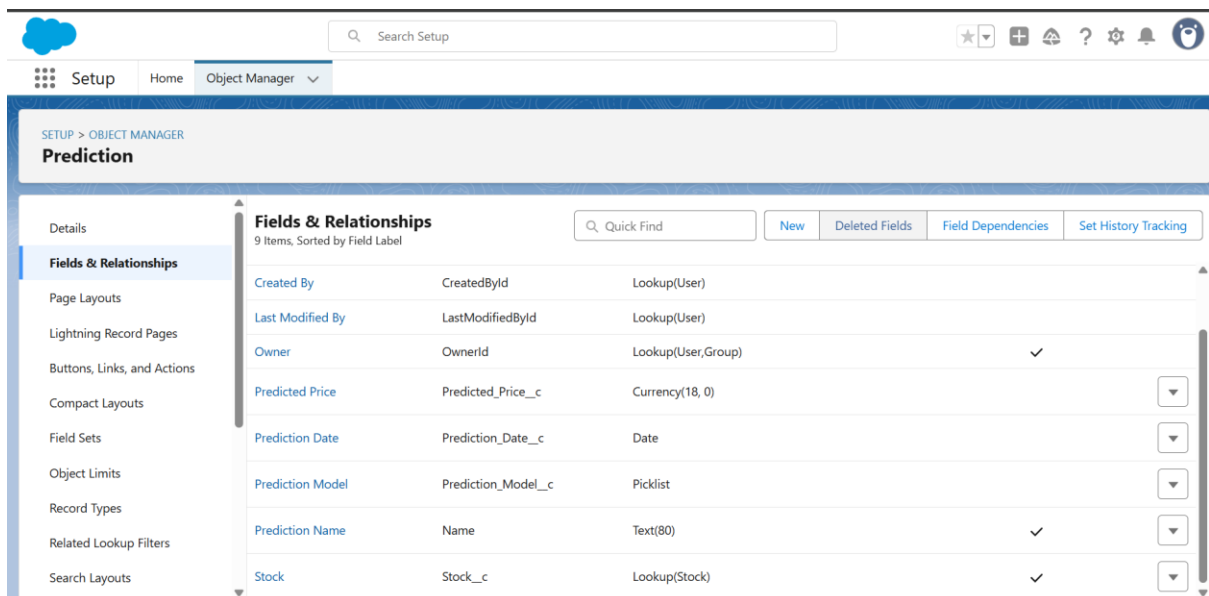
Track Field History
✓

Deployment Status
Deployed

Help Settings
Standard salesforce.com Help Window

Edit Delete

- Fields:
- Stock__c (Lookup)
- Predicted_Price__c (Currency)
- Confidence_Score__c (Percent)
- Prediction_Date__c (Date)
- Prediction_Model__c (Picklist: Moving Avg, ML, AI Service, etc.)



Setup > OBJECT MANAGER

Prediction

Fields & Relationships

9 Items, Sorted by Field Label

Quick Find

New Deleted Fields Field Dependencies Set History Tracking

Created By	CreatedBy	Lookup(User)	
Last Modified By	LastModifiedBy	Lookup(User)	
Owner	OwnerId	Lookup(User,Group)	✓
Predicted Price	Predicted_Price__c	Currency(18, 0)	▼
Prediction Date	Prediction_Date__c	Date	▼
Prediction Model	Prediction_Model__c	Picklist	▼
Prediction Name	Name	Text(80)	✓ ▼
Stock	Stock__c	Lookup(Stock)	✓ ▼

2.Relationships

🚦 **Portfolio__c → Transaction__c → Stock__c**

🔗 Many-to-Many via Transaction__c. One portfolio can hold many stocks, and one stock can belong to many portfolios.

🚦 **Stock__c → Prediction__c**

🔗 One-to-Many. A stock can have multiple predictions over time.

🚦 **User (Investor) → Portfolio__c (One-to-Many).**

3.Schema Builder (Visualization)

- **Investor (User) → Portfolio__c → Transaction__c → Stock__c**
- **Stock__c → Prediction__c**

🔗 This way:

- Admins see all data.
- Investors see **only their portfolios & related transactions**.
- Predictions link back to **stocks** for insights.

Phase 4: Process Automation (Admin)

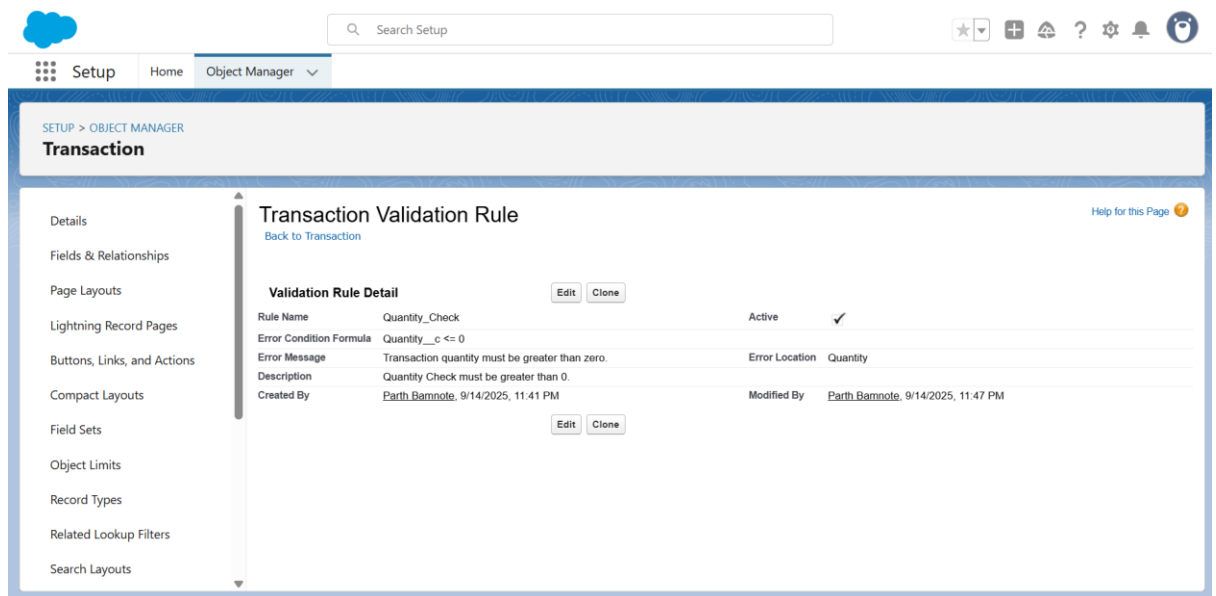
1. Validation Rules

✓ Prevent bad data entry.

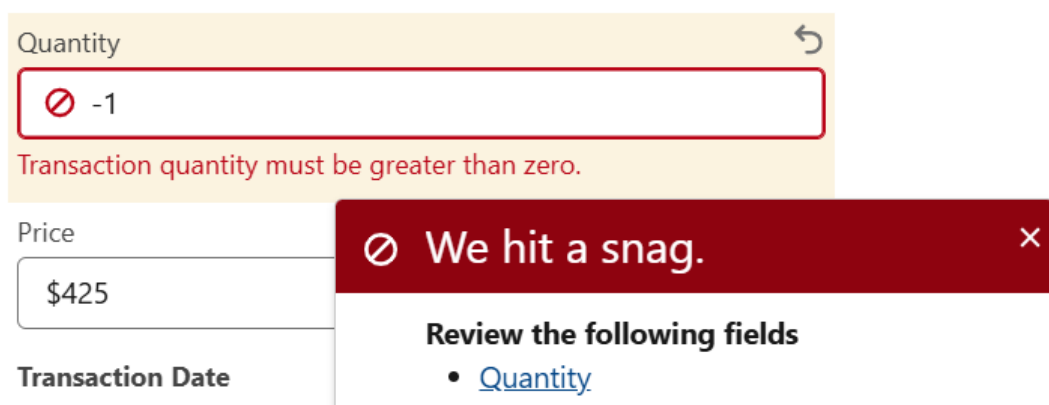
- $\text{Transaction_c.Quantity_c} > 0$
- Formula:

$\text{Quantity_c} \leq 0$

→ **Error Message: “Transaction quantity must be greater than zero.”**



The screenshot shows the Salesforce Setup interface. The left sidebar contains navigation links: Setup, Home, and Object Manager. The main content area is titled 'Transaction' and shows a 'Transaction Validation Rule' configuration page. The rule is named 'Quantity_Check' and is active. The error condition formula is $\text{Quantity_c} \leq 0$. The error message is 'Transaction quantity must be greater than zero.' The error location is 'Quantity'. The description is 'Quantity Check must be greater than 0.' The rule was created by 'Parth Bamnote' on 9/14/2025 at 11:41 PM and was last modified by the same user at the same time. There are 'Edit' and 'Clone' buttons for the rule.



The screenshot shows a form with three fields: 'Quantity', 'Price', and 'Transaction Date'. The 'Quantity' field contains the value '-1' and is highlighted with a red border. Below the field, a red error message is displayed: 'Transaction quantity must be greater than zero.' A red banner at the bottom of the form reads 'We hit a snag.' and lists the fields to review: 'Quantity'.

2. Flow (Record-Triggered)

⚡ Type: After-Save Flow on Transaction__c

Trigger: When a new transaction is inserted or updated.

Logic:

Get all transactions related to the portfolio.

Calculate :

SUM(Quantity__c * Price__c).

Update :

Portfolio__c.Total_Value__c.

💡 This keeps portfolio valuation auto-updated in real-time.

 **Apex Triggers**

Apex Trigger Detail

EditDeleteDownloadShow Dependencies

Name	TransactionTrigger	sObject Type	Transaction
Code Coverage	0% (0/28)	Status	Active
Created By	Parth Bamnote, 9/15/2025, 12:03 AM	Last Modified By	Parth Bamnote, 9/15/2025, 12:03 AM
Namespace Prefix			

Apex Trigger

Version Settings

Trace Flags

```
1trigger TransactionTrigger on Transaction__c (after insert, after update, after delete, after undelete) {
2
3  // Step 1: Collect impacted Portfolio Ids
4  Set<Id> portfolioIds = new Set<Id>();
5
6  if(Trigger.isInsert || Trigger.isUpdate || Trigger.isUndelete){
7      for(Transaction__c t : Trigger.new){
8          if(t.Portfolio__c != null){
9              portfolioIds.add(t.Portfolio__c);
10         }
11     }
12 }
13
14 if(Trigger.isUpdate || Trigger.isDelete){
15     for(Transaction__c t : Trigger.old){
16         if(t.Portfolio__c != null){
17             portfolioIds.add(t.Portfolio__c);
18         }
19     }
20 }
```

```

18     }
19 }
20 }
21
22 if(portfolioIds.isEmpty()){
23     return;
24 }
25
26 // Step 2: Query related Transactions
27 Map<Id, Decimal> portfolioTotals = new Map<Id, Decimal>();
28 for(Transaction__c txn : [
29     SELECT Id, Portfolio__c, Quantity__c, Price__c
30     FROM Transaction__c
31     WHERE Portfolio__c IN :portfolioIds
32 ]){
33     if(!portfolioTotals.containsKey(txn.Portfolio__c)){
34         portfolioTotals.put(txn.Portfolio__c, 0);
35     }
36     Decimal currentTotal = portfolioTotals.get(txn.Portfolio__c);
37     currentTotal += (txn.Quantity__c != null ? txn.Quantity__c : 0) *
38         (txn.Price__c != null ? txn.Price__c : 0);
39     portfolioTotals.put(txn.Portfolio__c, currentTotal);
40 }
41
42 // Step 3: Update Portfolios
43 List<Portfolio__c> portfoliosToUpdate = new List<Portfolio__c>();
44 for(Id pld : portfolioIds){
45     Portfolio__c p = new Portfolio__c(Id = pld);
46     p.Total_Value__c = portfolioTotals.containsKey(pld) ? portfolioTotals.get(pld) : 0;
47     portfoliosToUpdate.add(p);

```

```

31     WHERE Portfolio__c IN :portfolioIds
32 ]){
33     if(!portfolioTotals.containsKey(txn.Portfolio__c)){
34         portfolioTotals.put(txn.Portfolio__c, 0);
35     }
36     Decimal currentTotal = portfolioTotals.get(txn.Portfolio__c);
37     currentTotal += (txn.Quantity__c != null ? txn.Quantity__c : 0) *
38         (txn.Price__c != null ? txn.Price__c : 0);
39     portfolioTotals.put(txn.Portfolio__c, currentTotal);
40 }
41
42 // Step 3: Update Portfolios
43 List<Portfolio__c> portfoliosToUpdate = new List<Portfolio__c>();
44 for(Id pld : portfolioIds){
45     Portfolio__c p = new Portfolio__c(Id = pld);
46     p.Total_Value__c = portfolioTotals.containsKey(pld) ? portfolioTotals.get(pld) : 0;
47     portfoliosToUpdate.add(p);
48 }
49
50 if(!portfoliosToUpdate.isEmpty()){
51     update portfoliosToUpdate;
52 }
53}

```

Phase 5: Apex Programming (Developer)

1.SOQL (Salesforce Object Query Language)

SOQL lets you fetch records from Salesforce objects (like SQL).

Example 1: Get all Stocks in IT Sector

```
1 public class Stock {
2     public static void demo()
3     {
4         List<Stock__c> itStocks = [
5             SELECT Name,Current_Price__c, Sector__c
6             FROM Stock__c
7             WHERE Sector__c = 'IT'
8         ];
9         System.debug(itStocks);
10    }
```

```
13:06:27:036    USER_DEBUG    [9]|DEBUG|(Stock__c:{Name=TCS, Current_Price__c=4532, Sector__c=IT, Id=a0AgL000001qbm5UAA})
```

Example 2: Get Predictions for a Particular Stock

```
List<Prediction__c> stockPredictions = [
SELECT Predicted_Price__c, Confidence_Score__c
FROM Prediction__c
WHERE Stock__c = 'a0AgL000001qbm5UAA'
ORDER BY Predicted_Price__c DESC
];

System.debug(stockPredictions);
}
```

13:06:27:058 USER_DEBUG [17]DEBUG|(Prediction__c:{Predicted_Price__c=50000, Confidence_Score__c=90, Id=a0DgL00000BkbPdUAJ})

2.Apex Triggers

Triggers allow you to run automation when a record is created, updated, or deleted.

```
1 trigger UpdatePortfolioValue on Transaction__c (after insert, after update) {
2     Set<Id> portfolioIds = new Set<Id>();
3
4     // Collect Portfolio IDs from new transactions
5     for (Transaction__c t : Trigger.new) {
6         portfolioIds.add(t.Portfolio__c);
7     }
8
9     // Fetch related portfolios
10    List<Portfolio__c> portfolios = [
11        SELECT Id, Total_Value__c,
12            (SELECT Quantity__c, Price__c FROM Transactions__r)
13        FROM Portfolio__c
14        WHERE Id IN :portfolioIds
15    ];
16
17    // Recalculate Total Value
18    for (Portfolio__c p : portfolios) {
19        Decimal total = 0;
20        for (Transaction__c t : p.Transactions__r) {
21            total += t.Quantity__c * t.Price__c;
22        }
23        p.Total_Value__c = total;
24    }
25
26    update portfolios;
27 }
28
```

✅ This trigger ensures whenever a buy/sell transaction happens, the portfolio value updates automatically.

Phase 6: User Interface Development (Frontend)

Now, we build LWC components that read from those objects and display predictions.

Prediction Dashboard LWC

predictionDashboard.js

```
force-app > main > default > lwc > predictionDashboard > JS predictionDashboard.js > PredictionDashboard > columns
1  import { LightningElement, wire } from 'lwc';
2  import getPredictions from '@salesforce/apex/StockController.getPredictions';
3
4  export default class PredictionDashboard extends LightningElement {
5      predictions;
6      error;
7
8      columns = [
9
10         { label: 'Date', fieldName: 'Prediction_Date__c', type: 'date' },
11         { label: 'Predicted Price', fieldName: 'Predicted_Price__c', type: 'currency' },
12         { label: 'Confidence (%)', fieldName: 'Confidence_Score__c', type: 'percent' }
13     ];
14
15     @wire(getPredictions)
16     wiredPredictions({ error, data }) {
17         if (data) {
18             this.predictions = data;
19             this.error = undefined;
20         } else if (error) {
21             this.error = error;
22             this.predictions = undefined;
23         }
24     }
25 }
26
```

predictionDashboard.html

```
1  <template>
2    <lightning-card title="📈 Stock Predictions">
3      <template if:true={predictions}>
4        <lightning-datatable
5          key-field="id"
6          data={predictions}
7          columns={columns}>
8        </lightning-datatable>
9      </template>
10
11     <template if:false={predictions}>
12       <p>No predictions available</p>
13     </template>
14   </lightning-card>
15 </template>
16
```

✅ This shows predictions in a table.

 Stock Predictions			
☐	Date	Predicted P...	Confidence ...
☐	Sep 15, 2025	\$50,000.00	9,000%
☐	Sep 16, 2025	\$5,000.00	9,000%

Phase 7: Integration & External Access

Tools Used

Excel/Google Sheets → your external system where stock details are entered.

Zapier → the “bridge” that connects Excel ↔ Salesforce.

Salesforce → where your Stock__c records are stored.

Step 1: Prepare Excel Sheet

Make an Excel sheet (or Google Sheet) with columns like:

Ticker → e.g., TCS, INFY

Sector → IT, Pharma, Banking

Current Price → 3500, 1200

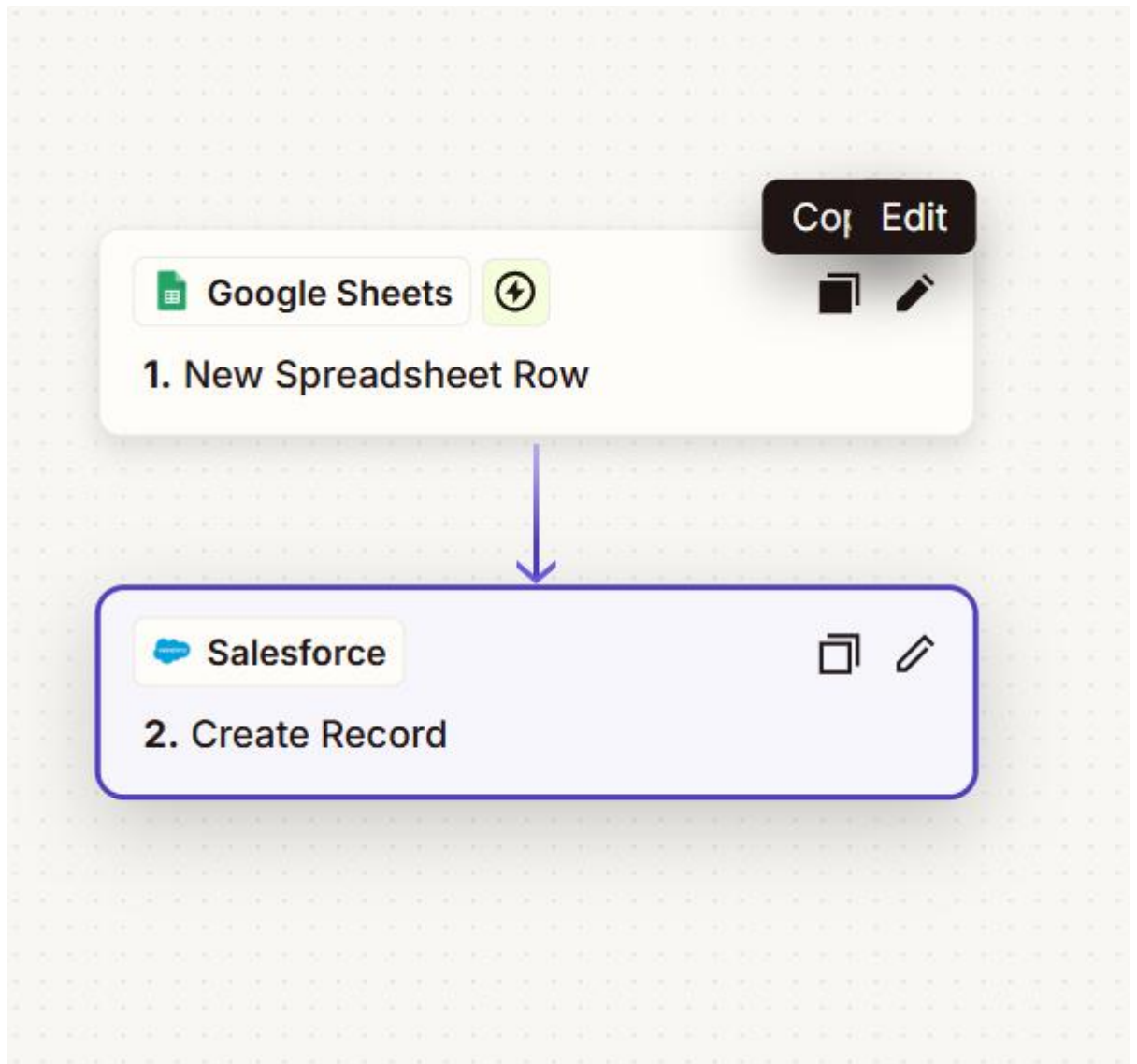
Market Cap → 1,200,000

Step 2: Connect Excel to Zapier

- In **Zapier**, choose **Excel (or Google Sheets)** → **New Row Added** as the **Trigger**.
- This means whenever you add a new stock in Excel, Zapier wakes up.

✅ Step 3: Connect Salesforce to Zapier

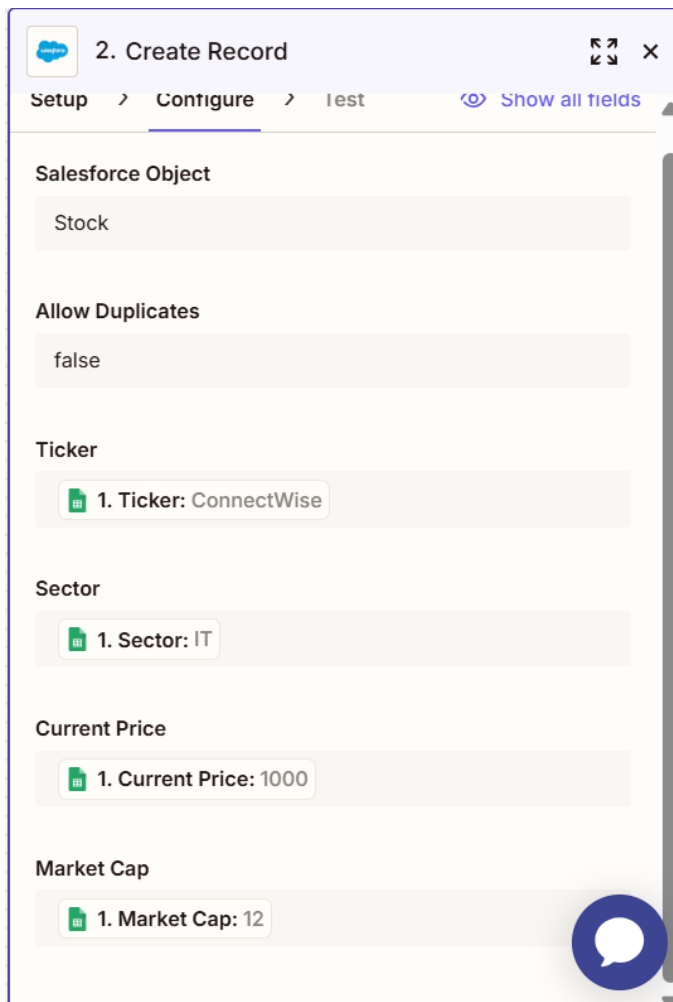
- Add an **Action** in Zapier → “Create Record in Salesforce.”
- Choose the **object** = **Stock__c**.



✅ Step 4: Map Excel Columns → Salesforce Fields

Tell Zapier how to fill Salesforce fields:

- Excel Ticker → Salesforce Ticker__c
- Excel Sector → Salesforce Sector__c
- Excel Price → Salesforce Current_Price__c
- Excel Market Cap → Salesforce Market_Cap__c



The screenshot shows the Zapier interface for creating a record in Salesforce. The title bar indicates '2. Create Record' with a close button. Below the title bar, there are tabs for 'Setup', 'Configure', and 'Test', with 'Configure' being the active tab. A 'Show all fields' link is visible. The main configuration area is divided into sections for mapping data from an Excel spreadsheet to Salesforce fields:

- Salesforce Object:** A dropdown menu showing 'Stock'.
- Allow Duplicates:** A dropdown menu showing 'false'.
- Ticker:** A dropdown menu showing '1. Ticker: ConnectWise'.
- Sector:** A dropdown menu showing '1. Sector: IT'.
- Current Price:** A dropdown menu showing '1. Current Price: 1000'.
- Market Cap:** A dropdown menu showing '1. Market Cap: 12'.

A blue circular button with a white speech bubble icon is located at the bottom right of the configuration area.

✅ Step 5: Test the Zap

- Add a row in Excel:

The screenshot shows a Zapier form for testing a Zap. It contains four input fields:

- Ticker:** SBI
- Sector:** Banking (selected from a dropdown menu)
- Current Price:** 50000
- Market Cap:** 6

At the bottom left is a purple **Submit** button, and at the bottom right is a [Clear form](#) link.

- Zapier sends it → Salesforce creates a new Stock__c record automatically.

✅ Step 6: Automate

- Keep Zapier running in the background.
- Every new row in Excel instantly creates a stock record in Salesforce.

	Ticker ↑	Sector	Market Cap	Current Price
1	ConnectWise	IT	12	\$1,000
2	Google	IT	12	\$13,468
3	Indigo	IT	12	\$2,345
4	SBI	Banking	6	\$50,000
5	TCS	IT	100	\$4,532

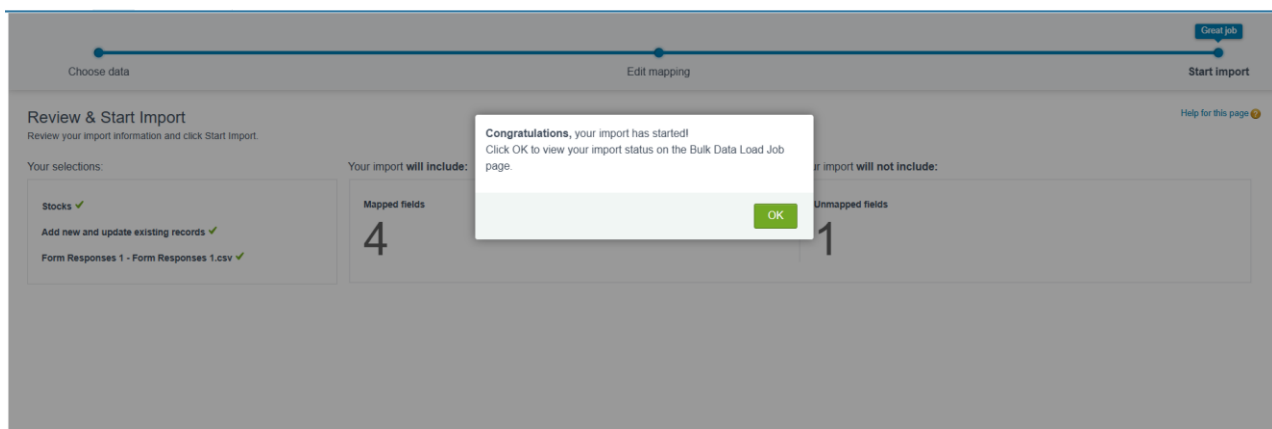
How this fits into Phase 7

- **Named Credentials / OAuth:** Zapier handles authentication to Salesforce (OAuth token).
- **External Access:** Excel is the external system; Salesforce is receiving data.
- **API Usage:** Zapier uses Salesforce APIs behind the scenes.
- **Change Data Capture / Platform Events:** Not needed here (Zapier is enough for simple stock registration).

Phase 8: Data Management & Deployment

1. Data Import Wizard (Beginner Tool)

- What: Salesforce built-in tool for uploading simple data (like Excel/CSV).
- Example: Import all Stocks from Excel (Ticker, Sector, Price) into Stock__c.



2.Data Loader (For Bulk Data)

What: Desktop tool for big uploads/downloads (50,000+ records).

Example: Upload 10,000 Transactions (buy/sell history) at once.

How:

- Install Data Loader.
- Login with Salesforce.
- Choose “Insert” or “Update.”
- Upload CSV file.

3. Duplicate Rules

What: Prevents duplicate records.

Example: Stop adding same stock ticker twice (TCS should not be added again).

How:

- Setup → Duplicate Rules → Create → Select Stock__c.
- Matching Rule: If Ticker__c is the same, block or alert.

The screenshot shows the 'Duplicate Rules' configuration page in Salesforce. The rule is named 'Prevent Duplicate Stocks'. The 'Rule Name' is 'Prevent Duplicate Stocks'. The 'Object' is 'Stock'. The 'Record-Level Security' is 'Enforce sharing rules'. The 'Action On Create' is 'Allow'. The 'Action On Edit' is 'Allow'. The 'Alert Text' is 'Use one of these records?'. The 'Active' checkbox is checked. The 'Matching Rule' is 'Prevent Duplicate Stocks matching rule' and is marked as 'Mapped'. The 'Matching Criteria' is 'Stock: Name EXACT MatchBlank = TRUE'. The 'Created By' is 'Parth Bamnote' and the 'Modified By' is 'Parth Bamnote'.

The screenshot shows the 'New Stock' form in Salesforce. The 'Ticker' field is populated with 'TCS'. The 'Sector' field is set to '--None--'. The 'Current Price' and 'Market Cap' fields are empty. An orange warning message box is displayed over the form, stating: 'Similar Records Exist. This record looks like an existing record. Make sure to check any potential duplicate records before saving. View Duplicates'. The 'Owner' is 'Parth Bamnote'. The form has 'Cancel', 'Save & New', and 'Save' buttons at the bottom.

4. Unmanaged vs Managed Packages

- Unmanaged Package → For learning/testing (can edit everything).
- Managed Package → For AppExchange apps (locked, versioned).
- Example: If you build a stock app just for learning → Unmanaged. If you want to publish to AppExchange → Managed.